

Lab Manual: Mounting External Cloud Storage in Databricks (Using Console)

Objective

By the end of this lab, you will:

- Create a secret scope in Databricks via the console.
- Mount **Google Cloud Storage (GCS)**, **Amazon S3**, and **Azure Data Lake Storage (ADLS)** into Databricks.
- Verify and use mounted paths (`/mnt/...`) in your ETL pipelines.

Prerequisites

- Access to a **Databricks workspace** with an active cluster.
- Credentials for cloud storage:
 - GCS: Service account JSON key.
 - AWS: Access Key + Secret Key.
 - Azure: Storage account key.

Step 1: Create a Secret Scope (via Console)

1. In the Databricks workspace, go to the **left-hand menu** and click **Data**.
2. Click **Secrets** (under Governance).
3. Click **Create Secret Scope**.
4. Enter:
 - a. **Scope name:** `myscope`
 - b. **Manage principal:** select who can access (e.g., All Users).
5. Click **Create**.

You now have a secret scope named `myscope`.

Step 2: Add Secrets to the Scope

1. Go to your newly created **secret scope** (myscope).
2. Click **Add Secret**.
3. For GCS:
 - a. Key: gcs-key
 - b. Value: Paste your **service account key JSON**.
4. Repeat for:
 - a. AWS: aws-access (Access Key), aws-secret (Secret Key).
 - b. Azure: adls-key (Storage account key).

All credentials are now securely stored.

Step 3: Mount Google Cloud Storage (GCS)

Paste this code into a Databricks notebook cell:

```
dbutils.fs.mount(  
    source="gs://my-gcs-bucket",  
    mount_point="/mnt/gcs",  
    extra_configs={  
        "fs.gs.auth.service.account.json.keyfile":  
dbutils.secrets.get(scope="myscope", key="gcs-key")  
    }  
)
```

Your GCS bucket is now mounted at /mnt/gcs.

Step 4: Mount Amazon S3

```
dbutils.fs.mount(  
    source="s3a://my-s3-bucket",  
    mount_point="/mnt/s3",  
    extra_configs={  
        "fs.s3a.access.key": dbutils.secrets.get(scope="myscope",  
key="aws-access"),  
        "fs.s3a.secret.key": dbutils.secrets.get(scope="myscope",
```

```
key="aws-secret")
    }
)
```

Your S3 bucket is now mounted at `/mnt/s3`.

Step 5: Mount Azure Data Lake Storage (ADLS)

```
dbutils.fs.mount(
    source="abfss://my-container@myaccount.dfs.core.windows.net/",
    mount_point="/mnt/adls",
    extra_configs={
        "fs.azure.account.key.myaccount.dfs.core.windows.net":
dbutils.secrets.get(scope="myscope", key="adls-key")
    }
)
```

Your ADLS container is now mounted at `/mnt/adls`.

Step 6: Verify Mounts

```
dbutils.fs.ls("/mnt")
```

You should see:

- `/mnt/gcs`
- `/mnt/s3`
- `/mnt/adls`

Step 7: Use Mounted Paths in ETL

```
# Read from GCS
df_gcs = spark.read.csv("/mnt/gcs/sample_data.csv", header=True,
inferSchema=True)
```

```
# Write to S3
```

```
df_gcs.write.mode("overwrite").parquet("/mnt/s3/output_data")
```

```
# Read from ADLS
```

```
df_adls = spark.read.parquet("/mnt/adls/processed_data")
```

Your ETL pipelines can now read/write seamlessly without exposing credentials.

Verification Checklist

- Secret scope created in console.
- Secrets added (GCS, AWS, Azure).
- GCS mounted at `/mnt/gcs`.
- S3 mounted at `/mnt/s3`.
- ADLS mounted at `/mnt/adls`.
- Data successfully read/written via mounted paths.

Summary

In this lab, you:

- Created a **secret scope in the Databricks console**.
- Stored GCS, S3, and ADLS credentials securely.
- Mounted external storage into `/mnt/...` paths.
- Verified and used the mounts in Spark ETL pipelines.

Now your Databricks cluster can securely access multiple cloud storages without ever hardcoding keys in code.